

Spark and Databricks Documentation

- [Spark and Databricks Documentation](#)
 - [Introduction to Azure Databricks](#)
 - [Definition](#)
 - [Databricks Architecture](#)
 - [Databricks Cluster](#)
 - [Definition](#)
 - [Cluster Types](#)
 - [Cluster Configuration](#)
 - [Key Factors Influencing Pricing](#)
 - [Pricing Components](#)
 - [Cost Calculation Example](#)
 - [Steps to Calculate Costs:](#)
 - [Overview of Cluster Pools](#)
 - [Example of a Cluster Pool](#)
 - [Cluster Policies](#)
 - [Databricks Notebooks](#)
 - [Magic commands](#)
 - [Databricks Utilities](#)
 - [Accessing Azure Data Lake from Databricks](#)
 - [Overview](#)
 - [Session-Scoped and Cluster-Scoped Authentication](#)
 - [Additional Authentication Methods](#)
 - [Azure storage Explorer](#)
 - [Accessing Data from Azure Data Lake with Access Keys in Databricks](#)
 - [What are Access Keys?](#)
 - [Key Management](#)
 - [Accessing Data from Databricks](#)
 - [Spark Configuration](#)
 - [Example Spark Command:](#)
 - [Accessing Azure Data Lake Gen2 Using Shared Access Signatures \(SAS Tokens\)](#)
 - [What is a Shared Access Signature \(SAS\)?](#)
 - [SAS Token Use Cases:](#)
 - [Accessing Data from Databricks Using SAS Tokens](#)
 - [Required Spark Configuration Parameters](#)
 - [Example Spark Configuration:](#)
 - [Accessing Azure Data Lake Storage Using Azure Service Principal](#)
 - [What is a Service Principal?](#)
 - [Steps to Access Data Lake with a Service Principal](#)
 - [Example Spark Configuration for Service Principal](#)
 - [Cluster Scope Authentication in Azure Databricks](#)
 - [What is Cluster Scoped Authentication?](#)
 - [How it Works](#)
 - [Example: Setting Spark Configuration in Cluster Creation](#)

- Azure Active Directory (AAD) Credential Pass-through Authentication
 - What is AAD Credential Pass-through Authentication?
 - How It Works:
- Securing Access to Azure Data Lake
 - Types of Secret Scopes
 - Implementation Steps
 - Key Differences
 - Summary
- Mounting Data Lake Container to Databricks
 - Databricks File System (DBFS)
 - DBFS Overview
 - DBFS Root
 - File Store
 - Managed Tables
 - Caution
 - Use Cases
 - Databricks Mount and Azure Storage
 - Benefits of Databricks Mounts
 - Transition to Unity Catalog
 - Unity Catalog Features
 - Mounting Azure Data Lake Storage Gen2
- Spark
 - Spark Cluster Architecture
 - Spark Dataframe and Data Source API
 - Databricks Workflows
 - Filter, Aggregations and Join Transformations
 - Using SQL in Spark Applications
 - Usage
- Spark SQL
 - Creating Databases
 - Managed And External Tables
 - Creating a Managed Table
 - Creating an External Table
 - Creating views
 - Local
 - Global
 - Creating Tables from a source
 - SQL Window Functions
- Data Loading Design Patterns
 - Full Load
 - Incremental Load
 - Hybrid Scenarios
- Delta Lake
 - Delta Lake and Data Architecture Evolution
 - Data Warehouse Challenges
 - Data Lake Benefits and Pitfalls

- [Delta Lake and Lakehouse Architecture](#)
- [Delta Lake Advanced Features](#)
- [Transaction Logs in Delta Lake](#)
- [Azure Data Factory](#)
 - [Introduction](#)
 - [Azure Data Factory Components](#)

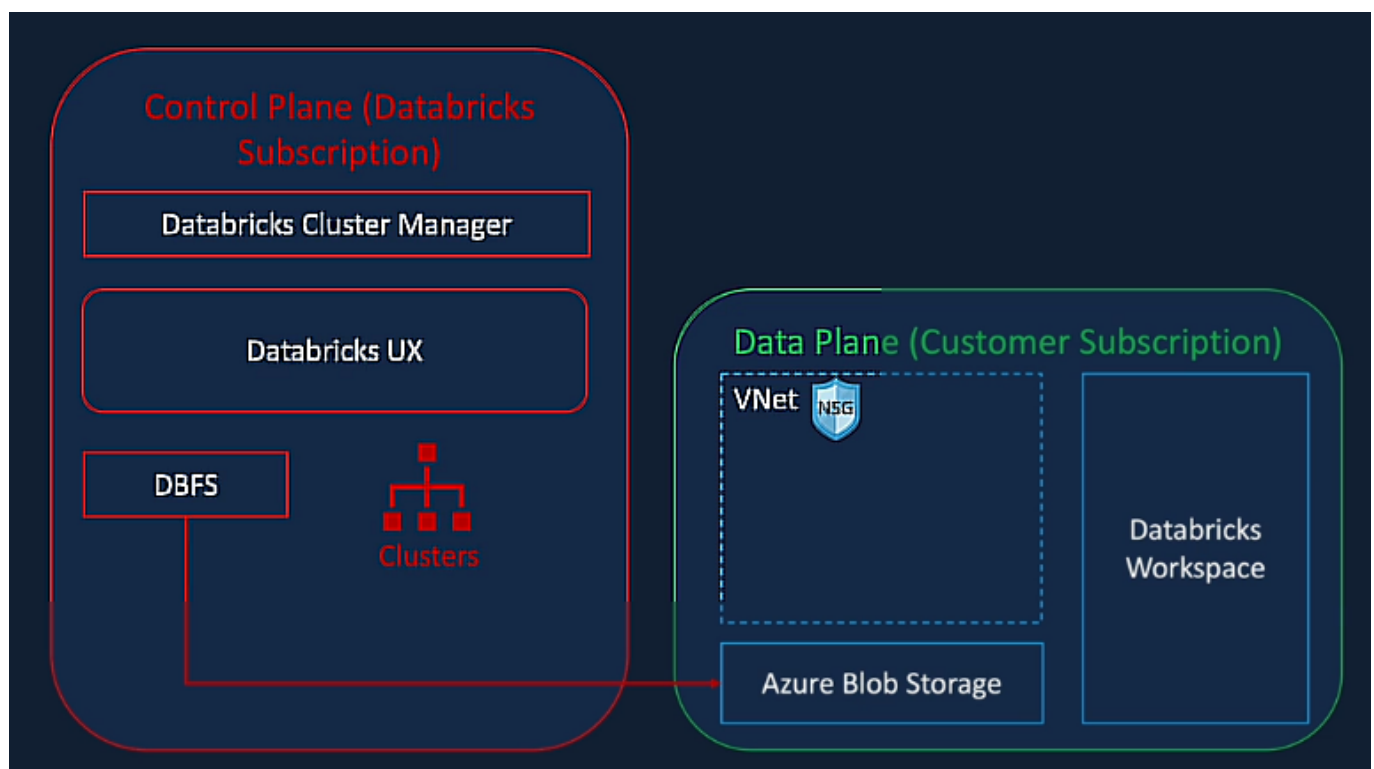
Introduction to Azure Databricks

Definition

Apache Spark is a fast unified analytics engine for big data processing and ML with an easy to use set of higher level of APIs. Databricks is a spark based unified data analytics platform that provides us with :

- setting up clusters
- managing security
- writing our code
- creating databases
- ACID transactions capability : Delta Lake
- SQL based analytics environment
- ML flow

Databricks Architecture



- **Control Plane:** The Control Plane handles orchestration, cluster management, and interface-related tasks but does not process or store customer data. It handles:
 - Cluster Management
 - Databricks UX (User Experience): web interface
 - Databricks File System (DBFS): metadata about storage

- Data Plane:
 - Virtual Network (VNET): for secure communication between resources.
 - Network Security Group (NSG): controls network traffic to and from resources.
 - Azure Blob Storage: used as default storage (DBFS) for temporary outputs.
 - Databricks Workspace: the workspace where the clusters computations happen.

Users access Databricks via Azure Active Directory Single Sign-On.

When a cluster is requested, Databricks' Cluster Manager provisions VMs in your VNet using Azure Resource Manager.

Data is processed and stored securely within your Azure subscription.

Databricks Cluster

Definition

A cluster is a collection of VMs. There is usually a driver node that orchestrates the tasks performed by worker nodes.

Cluster Types

Feature	All Purpose Clusters	Job Clusters
Creation	Created manually via GUI, CLI, or API	Created automatically when a job starts
Lifecycle	Persistent, can be terminated and restarted	Terminated at the end of the job, cannot be restarted
Usage	Suitable for interactive and ad-hoc analysis workloads	Suitable for automated workloads (e.g., ETL, ML workflows)
User Sharing	Can be shared among many users, good for collaboration	Isolated for the job being executed
Cost	More expensive to run	Less expensive
Summary	Great for interactive analysis and ad-hoc work	Great for repeated production workloads

Cluster Configuration

Feature	Single Node Clusters	Multi Node Clusters
Node Configuration	Consists of one Driver Node (no Worker Nodes)	One Driver Node and one or more Worker Nodes
Scalability	Not horizontally scalable	Horizontally scalable; additional Worker Nodes can be added
Use Cases	Lightweight ML and data analysis	Suitable for large Spark Jobs and workloads

Feature	Single Node Clusters	Multi Node Clusters
Access Modes	Single User access only	Supports Single User, Shared, No Isolation Shared, and Custom access modes
Shared Access	Not applicable	Shared access available with and without process isolation
Cluster Runtimes	Not applicable	Includes Databricks Runtime, ML Runtime, Photon Runtime, and Light Runtime
Auto Termination	Not applicable	Automatically shuts down idle clusters (10-10,000 minutes)
Auto Scaling	Not applicable	Automatically adjusts Worker Nodes based on workload
VM Types	Not applicable	Memory, Compute, Storage, General Purpose, and GPU Accelerated instance types available
Cluster Policies	Not applicable	Simplifies configuration and controls costs

Key Factors Influencing Pricing

Factor	Description
Type of Workload	Different workloads (e.g., All-Purpose Compute, Jobs Compute, Databricks SQL) have varying costs.
Workspace Tier	Databricks offers Standard and Premium tiers; Premium is more expensive but offers more features.
Virtual Machine Type	The type of VM impacts pricing (e.g., GPU-enabled VMs are more expensive than General Purpose VMs).
Pre-purchase Plans	Azure provides discounts for pre-purchasing compute capacity in advance.

Pricing Components

1. **Databricks Units (DBUs):**
 - DBUs are a normalized measure of processing power. Each cluster uses a certain number of DBUs, which depends on the type of workload and workspace tier.
2. **Virtual Machines:**
 - You also need to account for the cost of the virtual machines used in the cluster. This includes both the driver node and any worker nodes.

Cost Calculation Example

Using a single-node cluster as an example:

- **Databricks Units:** 0.75 DBUs.
- **VM Type:** Standard DS3_v2.
- **Pricing Tier:** Premium.
- **Workload Type:** All-Purpose Compute.
- **Pay-as-you-go Plan:** No discounts.

Steps to Calculate Costs:

1. Calculate DBU Cost:

- DBU cost = DBUs × Price per DBU.
- For example: $(0.75 \times 0.55 = 0.41, \text{approx. } \$0.41/\text{hour})$.

2. Calculate VM Costs:

- Cost of Driver node: $(\text{Price of Standard DS3_v2} = 0.35)$.
- Cost of Worker nodes: Zero (since it's a single-node cluster).

3. Total Cluster Cost:

- Total cost = DBU cost + Cost of Driver node + Cost of Worker nodes.
- For example: $(0.41 + 0.35 + 0 = 0.76, \text{approx. } \$0.76/\text{hour})$.

4. Extrapolate for Duration:

- For a cluster running for 10 hours: $(0.76 \times 10 = 7.60)$.

Overview of Cluster Pools

Clusters typically take time to start up and auto-scale. To minimize this time, we can use **Cluster Pools**. A Cluster Pool is essentially a set of idle, ready-to-use virtual machines that help reduce cluster start and auto-scaling times. It's important to not that having an idle node incurs charges.

Example of a Cluster Pool

- **Minimum Instances:** 1
- **Maximum Instances:** 2

This configuration means that the pool will always keep at least one instance ready for clusters to consume until it reaches the maximum limit of two instances.

1. **Cluster 1** requests one node:

- The pool provides the one instance and spins up another one to be ready.

2. **Cluster 2** requests one more node:

- The pool gives that instance to Cluster 2.

3. If **Cluster 2** requests two more nodes, it will fail to start with an error indicating there aren't enough instances available to allocate.

Cluster Policies

Cluster Policies enable administrators to hide unnecessary configuration options, fix certain parameters, and set default values, creating a streamlined user interface. By defining specific rules in JSON format, administrators can restrict users to single-node clusters, limit node types, and set auto termination. They can be created from scratch or modifying existing ones by overriding specific attributes.

Databricks Notebooks

Databricks offers a Jupyter style notebook, with some additional capabilities to carry out development that run commands on a databricks cluster.

Magic commands

They allow us to override the default language in a Notebook. Use % followed by the language of your choice.

Databricks Utilities

Databricks Utilities make it easier to combine different types of tasks in a Single notebook. For example, they allow us to combine file operations with ETL tasks. These utilities can only be run from Python, Scala or R cells in a Notebook.

Utility	Description
dbutils.fs	Provides file system utilities to access and manage data in cloud storage.
dbutils.notebook	Enables running other notebooks and managing workflows within Databricks.
dbutils.jobs	Allows interaction with jobs, including creating and managing them.
dbutils.secrets	Securely manages and retrieves secrets for accessing sensitive information.
dbutils.widgets	Creates interactive widgets to allow user input in notebooks.
dbutils.library	Manages libraries and dependencies in Databricks notebooks.
dbutils.fs.help()	Displays help documentation for the filesystem utilities.
dbutils.notebook.exit()	Exits a notebook and returns a value to the calling notebook.

Accessing Azure Data Lake from Databricks

Overview

Azure Databricks offers several ways to authenticate and access Azure Data Lake Storage Gen2 (commonly used as a data storage solution). These methods include:

- **Access Key** : Each Azure storage account has an access key, which can be used to authenticate and access the storage account.
- **Shared Access Signature (SAS)**: A SAS token provides a way to access the storage account with more granular control than an access key. It allows you to define specific permissions for accessing the storage account.

- **Service Principal:** You can create a Service Principal, assign necessary permissions to it for the Data Lake, and use these credentials to access the storage account securely.

Session-Scoped and Cluster-Scoped Authentication

There are two ways to use the above credentials in Databricks:

- **Session-Scoped Authentication** This involves using the credentials within a Databricks Notebook. Authentication is valid only for the duration of the session, i.e., until the notebook is detached from the cluster.
- **Cluster-Scoped Authentication** Here, the credentials are configured at the cluster level. Authentication takes place when the cluster starts and remains valid until the cluster is terminated. All notebooks attached to the cluster can access the data.

Additional Authentication Methods

- **Azure Active Directory (AAD) Pass-Through Authentication:** This option allows you to use Azure Active Directory credentials for authentication. The cluster checks the user's Azure roles and permissions assigned via Azure's Identity and Access Management (IAM). This feature is only available in premium workspaces.
- **Unity Catalog:** A more recent addition, Unity Catalog allows administrators to set access permissions for users. When a user tries to access the storage account, the cluster checks the Unity Catalog for the user's access rights. Like AAD Pass-Through Authentication, Unity Catalog is only available in premium workspaces.

Azure storage Explorer

Azure Storage Explorer is a free, standalone desktop application that allows users to easily manage Azure cloud storage resources. Thanks to its features, It is commonly advised to use in large projects.

Accessing Data from Azure Data Lake with Access Keys in Databricks

What are Access Keys?

When you create an Azure storage account, Azure generates two 512-bit storage account access keys. These keys provide full access to the storage account, meaning anyone with the key can perform all operations, just like an owner. Therefore, it's critical to secure these access keys properly.

Key Management

- Azure recommends storing access keys in **Azure Key Vault** for better security.
- In case a key is compromised, you can rotate or regenerate new keys.
- Having two keys allows uninterrupted access while one key is being rotated.

Accessing Data from Databricks

To access data from Azure Data Lake Storage (ADLS) Gen2 in Databricks, you'll need to configure the access key in Spark settings. This can be done using the following configuration:

Spark Configuration

The configuration parameter has two parts:

- **First part:** `fs.azure.account.key.<storage-account-endpoint>`
- **Second part:** The access key for your storage account.

Example Spark Command:

```
spark.conf.set(  
  "fs.azure.account.key.formula1dl.dfs.core.windows.net",  
  "<your-access-key>"  
)
```

Accessing Azure Data Lake Gen2 Using Shared Access Signatures (SAS Tokens)

What is a Shared Access Signature (SAS)?

SAS tokens allow more granular control over access compared to Access Keys. With SAS tokens, you can:

- Restrict access to specific **resource types** (e.g., Blob containers, queues, tables).
- Define **permissions** like Read-only access, preventing Write or Delete operations.
- Set a specific **time period** for access.
- Restrict access to specific **IP addresses**, ensuring security by preventing public access.

This level of control makes SAS tokens a recommended access pattern for external clients who shouldn't have full access to your storage account.

SAS Token Use Cases:

- Allowing **Read-only access** to a Blob container.
- Granting access to the storage account for a **limited time period**.
- **IP-based restrictions** to block unauthorized access.

Accessing Data from Databricks Using SAS Tokens

To authenticate Azure Databricks to access data in Azure Data Lake Storage Gen2 with a SAS token, you need to configure Spark settings, similar to how we used Access Keys in the previous lesson.

Required Spark Configuration Parameters

1. **Authentication Type:** Define the authentication type as `SAS` (Shared Access Signature).
2. **SAS Token Provider:** Set the SAS token provider to `Fixed SAS Token Provider`.
3. **SAS Token:** Provide the actual value of the SAS token.

Example Spark Configuration:

```
spark.conf.set("fs.azure.account.auth.type.<storage-account-name>.dfs.core.windows.net", "SAS")
spark.conf.set("fs.azure.sas.token.provider.type.<storage-account-name>.dfs.core.windows.net", "org.apache.hadoop.fs.azure.FixedSASTokenProvider")
spark.conf.set("fs.azure.sas.token.<storage-account-name>.dfs.core.windows.net", "<your-sas-token>")
```

Accessing Azure Data Lake Storage Using Azure Service Principal

Principals are similar to user accounts, but they are specifically designed for automated access scenarios like running jobs in **Azure Databricks** or in **CI/CD pipelines**.

What is a Service Principal?

A **Service Principal** is essentially an application registered in **Azure Active Directory (AD)**. It is given permissions to access resources in an Azure subscription using **role-based access control (RBAC)**. Service Principals are preferred in automated processes because they:

- Provide better **security** and **traceability**.
- Enable **audit** and **monitoring** of access and actions.
- Allow each application to have its own **Service Principal** with **specific permissions**.

Steps to Access Data Lake with a Service Principal

1. **Register the Service Principal:** First, you need to **register the Service Principal** in **Azure AD**, also referred to as an **Azure AD Application**. Once registered, it is assigned a unique identifier called the **Application/Client ID**.
2. **Create a Secret:** Next, create a **secret** for the Service Principal, which will serve as its password. This secret will be used in Databricks to authenticate the Service Principal.
3. **Configure Databricks:** You can configure **Azure Databricks** to access the storage account using the Service Principal by setting the appropriate **Spark configuration parameters**.
4. **Assign Permissions:** Now, assign the necessary role to the Service Principal. For this lesson, we'll use the **Storage Blob Data Contributor** role, which grants full access to the storage account. Alternatively, you can use **Storage Blob Data Reader** if only read-only access is needed.

Example Spark Configuration for Service Principal

```
spark.conf.set("fs.azure.account.auth.type.<storage-account-name>.dfs.core.windows.net", "OAuth")
spark.conf.set("fs.azure.account.oauth.provider.type.<storage-account-name>.dfs.core.windows.net", "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider")
```

```
spark.conf.set("fs.azure.account.oauth2.client.id.<storage-account-name>.dfs.core.windows.net", "<application-client-id>")
spark.conf.set("fs.azure.account.oauth2.client.secret.<storage-account-name>.dfs.core.windows.net", "<service-principal-secret>")
spark.conf.set("fs.azure.account.oauth2.client.endpoint.<storage-account-name>.dfs.core.windows.net", "https://login.microsoftonline.com/<tenant-id>/oauth2/token")
```

Cluster Scope Authentication in Azure Databricks

In the previous lessons, we authenticated from **Databricks Notebooks** by setting **Spark configuration parameters** with secrets like **Access Keys** or **SAS tokens**. This process was specific to the **session** created when the notebook was attached to a **Databricks Cluster**. Once the notebook was detached, the authentication expired, and re-authentication was required for further access to the storage account.

What is Cluster Scoped Authentication?

In **Cluster Scoped Authentication**, we move the authentication process from the **notebook** to the **cluster configuration**. Instead of setting the secrets and Spark configurations within each notebook, we configure them **directly on the cluster** at startup.

Once the cluster is created, every notebook that connects to the cluster will **automatically have access** to the **Azure Data Lake**.

How it Works

- During the **cluster creation**, Spark configuration parameters (along with the necessary secrets) are set in the **cluster configuration**.
- The cluster completes the authentication when it starts up, making the credentials available to every notebook connected to it.

Example: Setting Spark Configuration in Cluster Creation

You would set similar Spark configurations for **Access Keys** or **SAS tokens** directly in the **cluster configuration**:

```
spark.conf.set("fs.azure.account.key.<storage-account-name>.dfs.core.windows.net", "<access-key>")
```

Azure Active Directory (AAD) Credential Pass-through Authentication

While the previous methods are effective, they share a common limitation: **all users with the secret** and access to the same **Databricks cluster** have the **same level of access** to the storage account.

In certain scenarios, you might want to restrict access for specific users based on their roles.

What is AAD Credential Pass-through Authentication?

AAD Credential Pass-through allows **Databricks** to authenticate to **Azure Data Lake Storage (ADLS)** using the **individual user's Azure Active Directory (AAD) credentials**. This means that each user's access to a storage account is determined by their **Role-Based Access Control (RBAC) roles** in **Azure Active Directory**.

How It Works:

1. When a user runs a notebook on a **Databricks cluster**, their **AAD credentials** are passed through to the **ADLS** storage account.
2. Databricks verifies whether the user has the appropriate **RBAC role** assigned to the storage account.

Securing Access to Azure Data Lake

Storing access keys and secrets in clear text within notebooks or clusters can lead to security vulnerabilities. **Secret Scopes** are collections of secrets that provide a secure way to store credentials and reference them in notebooks, clusters, and jobs.

Types of Secret Scopes

1. **Databricks Backed Secret Scope:**
 - Managed by Databricks and uses an encrypted database to store secrets.
 - Can be created or modified using the Databricks CLI or API, but not through the GUI.
2. **Azure Key Vault Backed Secret Scope:**
 - Recommended approach for storing secrets when using Databricks on Azure.
 - Allows secrets to be shared with other Azure services (e.g., Azure Data Factory, Azure Synapse Analytics).
 - Provides centralized management of secrets.

Implementation Steps

1. **Create an Azure Key Vault:**
 - This will serve as the central location for storing all secrets.
2. **Add Secrets to the Key Vault:**
 - Store your credentials securely in the Azure Key Vault.
3. **Create a Databricks Secret Scope:**
 - Link this scope to the Azure Key Vault to enable secure access to the secrets.
4. **Use the Databricks Secrets Utility:**
 - Utilize `dbutils.secrets` to retrieve secrets from the linked secret scope in notebooks, clusters, or jobs.

Key Differences

Feature	Secret Scope	Azure Key Vault
---------	--------------	-----------------

Feature	Secret Scope	Azure Key Vault
Definition	A way to store and manage secrets in Databricks.	A secure cloud service for storing sensitive information.
Storage	Backed by Databricks' encrypted database or linked to Key Vault.	A dedicated Azure service for secret management.
Use Case	Primarily for managing credentials in Databricks notebooks and jobs.	Can be used across various Azure services for shared secret management.
Access	Accessed via Databricks Secrets Utility (e.g., <code>dbutils.secrets.get(...)</code>).	Accessed through Azure APIs, SDKs, or linked to Secret Scopes in Databricks.
Functionality	Simplifies secret management specifically for Databricks.	Provides advanced features like logging and access policies.
Management	Easier to set up within Databricks.	Offers comprehensive secret management capabilities.

Summary

- **Secret Scopes** are used to easily access **Azure Key Vault** secrets in Databricks.
- **Azure Key Vault** serves as a centralized secret management system usable by multiple Azure services, while **Secret Scopes** simplify access specifically for Databricks workloads.

Mounting Data Lake Container to Databricks

Databricks File System (DBFS)

DBFS Overview

Databricks File System (DBFS) is a distributed file system that provides access to data stored in Azure Blob Storage, which is mounted by default when a Databricks workspace is created.

DBFS Root

The default storage area in DBFS is called DBFS Root. It allows interaction with Azure Blob Storage and is accessible from any Databricks Cluster.

File Store

DBFS includes a special folder called File Store, used for temporary storage (e.g., images, small datasets) accessible through the Databricks UI.

Managed Tables

Managed tables created without a specified location default to DBFS Root, but this can be changed during creation.

Caution

It is not advisable to store customer data in DBFS Root since it will be deleted if the workspace is dropped. Instead, use an external Data Lake for customer data.

Use Cases

DBFS is suitable for quick analyses and temporary storage of files needed in notebooks, but files in the File Store are accessible to all users with workspace access.

Databricks Mount and Azure Storage

Customers can create separate Azure Blob Storage or Azure Data Lake Storage accounts within their subscription to store data. This architecture ensures that when you delete the Databricks workspace, the customer data remains intact.

To access the storage, we can use the **Azure Blob File System (ABFS) protocol**, similar to what we did in the previous section of the course. However, this approach has two drawbacks:

1. **Complex URLs:** Accessing files requires dealing with long ABFS URLs rather than simple file system semantics.
2. **Authentication:** Each time you access the storage accounts, you must use credentials for authentication.

Benefits of Databricks Mounts

Databricks allows you to mount storage accounts to DBFS. Here are the benefits of using Databricks mounts:

- **No Credentials Required:** Once mounted, everyone with workspace access can access the data without providing credentials.
- **File System Semantics:** Users can interact with storage using familiar file system commands instead of long URLs.

Comparison: DBFS serves as an abstraction layer while still utilizing Azure storage, benefiting from its performance tiers, replication, and massive storage capabilities.

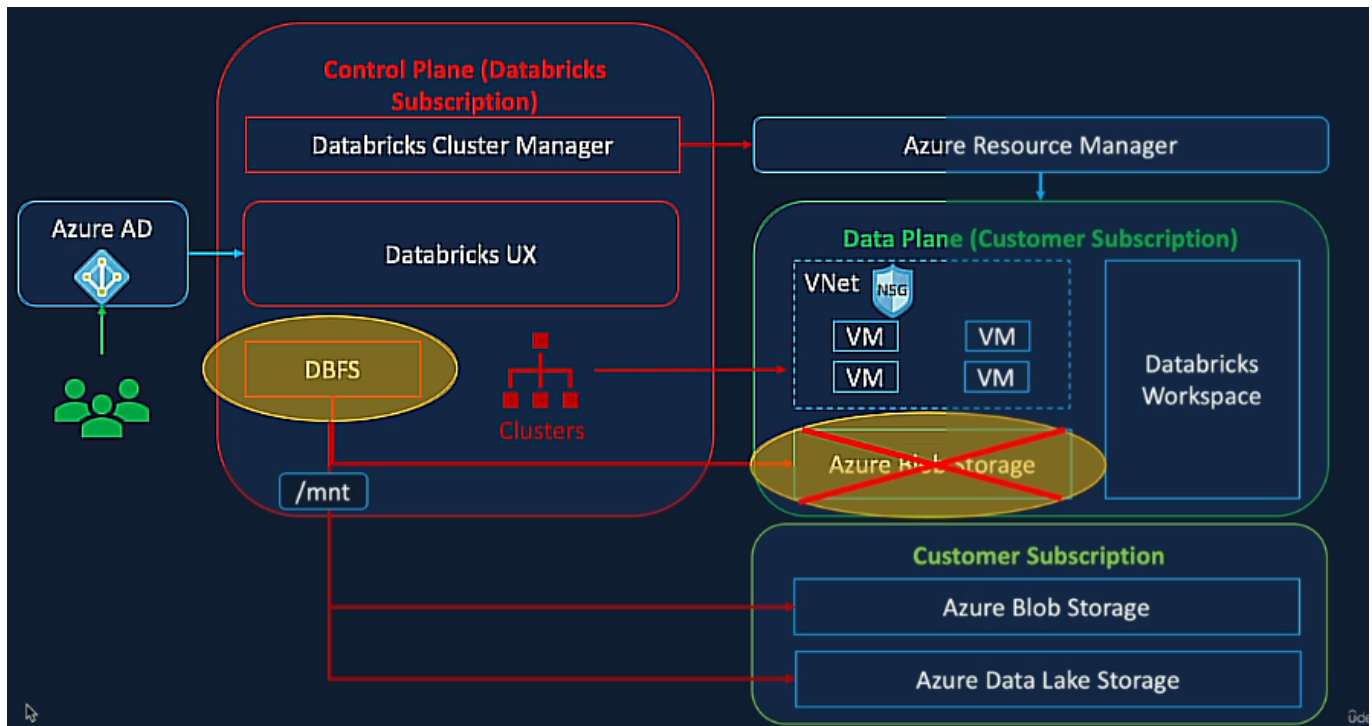
Transition to Unity Catalog

Databricks initially recommended using mounts to access Azure Data Lake. However, with the introduction of **Unity Catalog** (generally available since late 2022), Databricks now suggests using Unity Catalog for enhanced security.

Unity Catalog Features

- **No Credentials Needed:** When using Unity Catalog, you can access the Data Lake with the ABFS protocol without specifying credentials.
- **Centralized Access:** Unity Catalog provides centralized access across multiple Databricks workspaces, making it ideal for large projects.

Mounting Azure Data Lake Storage Gen2



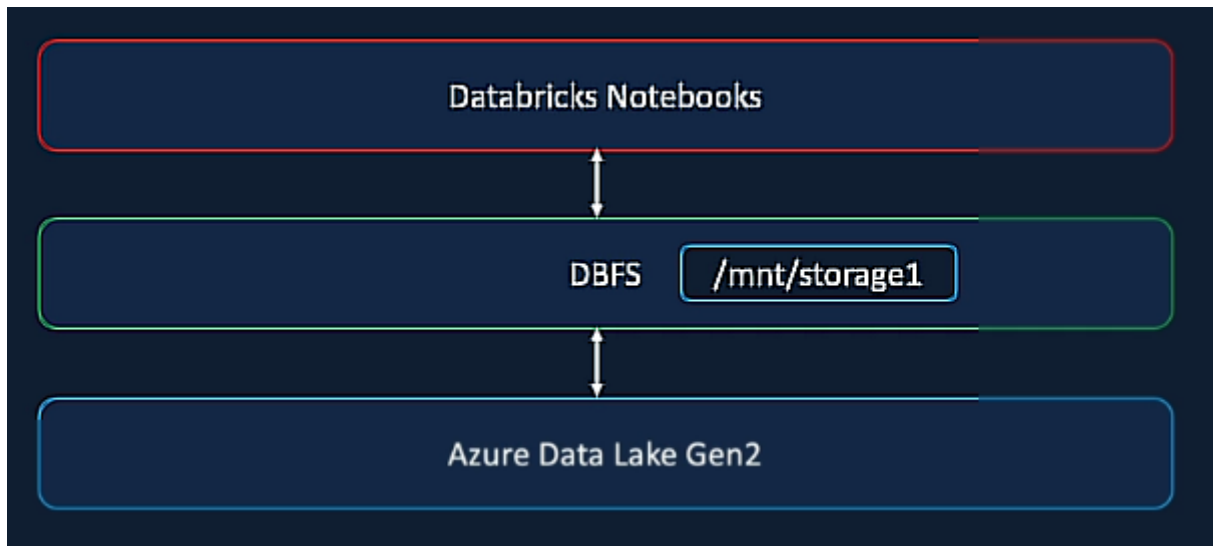
To mount Azure Data Lake Storage Gen2 to the Databricks File System (DBFS), follow these steps:

1. **Create a Service Principal:** A Service Principal is an Azure Active Directory credential similar to a user account, acting as a service account.
2. **Grant Access:** Provide the Service Principal access to the Data Lake storage.
3. **Create Mount Points:** Use the Service Principal's credentials to create mount points in DBFS.

```
configs = {"fs.azure.account.auth.type": "OAuth",
           "fs.azure.account.oauth.provider.type":
           "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider",
           "fs.azure.account.oauth2.client.id": "<application-id>",
           "fs.azure.account.oauth2.client.secret": dbutils.secrets.get(scope="
<scope-name>",key="<service-credential-key-name>"),
           "fs.azure.account.oauth2.client.endpoint":
           "https://login.microsoftonline.com/<directory-id>/oauth2/token"}

# Optionally, you can add <directory-name> to the source URI of your mount point.
dbutils.fs.mount(
    source = "abfss://<container-name>@<storage-account-
name>.dfs.core.windows.net/",
    mount_point = "/mnt/<mount-name>",
    extra_configs = configs)
```

These mount points allow you to access storage without needing to specify credentials and enable the use of file system semantics (e.g., `/mnt/storage1`).



Spark

Spark Cluster Architecture

Databricks mainly uses PySpark and Spark SQL for projects. Knowing about Spark architecture helps improve skills as a Spark developer.

- **Cluster Structure:** A Databricks cluster has one Driver node and multiple Worker nodes. Each node is a virtual machine in Azure. The Driver node runs a program that manages resources and tasks but doesn't do the actual work.
- **Executor Configuration:** Each Worker node has one executor running on its own Java Virtual Machine (JVM). This setup is easy to manage and efficient. Executors do the data processing and handle reading and writing data from other sources. They have slots for tasks, equal to the number of cores in the Worker nodes.
- **Task Management:** An application breaks down into jobs, stages, and tasks. Tasks are the basic units that get done. The Driver assigns tasks to executors, which complete the work and send the results back to the Driver.
- **Scaling Clusters:**
 - **Vertical Scaling:** Increasing the number of cores in a single node (like going from four to six cores), limited by what a virtual machine can handle.
 - **Horizontal Scaling:** Adding more Worker nodes to the cluster, which allows it to handle larger data sets, including petabytes.

Spark Dataframe and Data Source API

Spark provides Data Sources API with DataFrame reader methods to read the data from a number of data formats including CSV, JSON, ORC, Parquet, etc. Each DataFrame is then divided into logical partitions, which could then be computed by different executors or slots.

This gives Spark the ability to divide the work and create smaller tasks and execute them in parallel to get the results quickly.

Databricks Workflows

Spark lets us reuse a notebook in another notebook using the `%run` magic command.

It is possible to pass parameters in our notebooks by using `dbutils.widgets`. To orchestrate jobs workflow, databricks offers the notebook utility `dbutils.notebook.exit` for example. However, this is not the best approach.

Filter, Aggregations and Join Transformations

Spark offers us several methods for data manipulation and analysis, enabling efficient processing and querying of large datasets.

Transformation Type	Description	Methods
Filter	Used to remove unwanted rows from a dataset based on specific conditions.	<code>filter(condition)</code> <code>where(condition)</code>
Aggregations	Performs calculations on the data, such as summing, counting, or averaging.	<code>groupBy(column)</code> <code>agg(*[functions])</code>
Join	Combines rows from two or more DataFrames based on a common key or condition.	<code>join(otherDataFrame, on, how)</code>

Using SQL in Spark Applications

In order to access dataframes using SQL, we create Views.

View Type	Description	Scope
Local Temp View	A temporary view that is only accessible within the session where it was created. It does not persist after the session ends.	Session-scoped; cannot be accessed from other sessions.
Global Temp View	A temporary view that is accessible across all sessions. It persists until the Spark application terminates.	Application-scoped; can be accessed from any session.

Usage

- **Local Temp View:** Use when a view is needed only for a specific session, e.g., for intermediate computations or analysis.
`df.createOrReplaceTempView("local_temp_view")`
- **Global Temp View:** Use when a view needs to be shared among multiple sessions, e.g., for collaborative analysis or reporting.
`df.createOrReplaceGlobalTempView("global_temp_view")`

Spark SQL

Creating Databases

We can create databases and mount them to a specific location for easier access.

```
CREATE DATABASE IF NOT EXISTS database_name
LOCATION "/location"

DESCRIBE EXTENDED DATABASE database_name
```

Managed And External Tables

Table Type	Description	Storage Location
Managed Table	Spark manages both the data and the metadata. When the table is dropped, both the data and metadata are deleted.	Stored in the default warehouse location (<code>spark.sql.warehouse.dir</code>) unless a different path is specified.
External Table	Spark manages only the metadata. The actual data is stored externally, and when the table is dropped, only the metadata is removed, not the data.	Stored at the user-specified location. Data remains intact even if the table is dropped.

Creating a Managed Table

```
CREATE TABLE managed_table (
  id INT,
  name STRING
)
USING PARQUET;
```

Creating an External Table

```
CREATE TABLE external_table (
  id INT,
  name STRING
)
USING PARQUET
LOCATION '/mnt/data/external_table';
```

Creating views

Local

```
dataframe.createOrReplaceTempView("local_view_name")
```

```
SELECT * FROM local_view_name;
```

Global

```
dataframe.createOrReplaceGlobalTempView("global_view_name")
```

```
SELECT * FROM global_temp.global_employee_view;
```

Creating Tables from a source

We can create tables from various files(csv, json, multi files) using the `sql OPTION` comand. We can also save tables under parquet format using `.format("parquet").saveAsTable("/location")` .

SQL Window Functions

It works similarly to spark dataframe windows. In this example we are using the rank function:

```
SELECT field_name, RANK() OVER(PARTITION BY field1_name ORDER BY field2_name ) as  
rank  
FROM ...  
-- rest of code --
```

Data Loading Design Patterns

Full Load

A **full load** involves processing all the data received from the source system every time. This is suitable for smaller datasets but becomes inefficient with larger datasets due to the need to reprocess everything. The system processes all the data each time, regardless of whether most of it has changed.

Incremental Load

An **incremental load** processes only the data that has changed since the last load. This approach is more efficient for large datasets. This reduces the processing time by only handling new or modified data, minimizing the overall workload.

Hybrid Scenarios

In real-world projects, a combination of full and incremental loads might be used:

- You can receive a full dataset but process it incrementally to improve performance.
- You can receive incremental data but process it fully for specific use cases, such as calculating aggregates across all data.

Delta Lake

Delta Lake and Data Architecture Evolution

Delta Lake is an open-source storage layer created by Databricks to bring reliability and performance to Data Lakes. It supports ACID transactions, scalable metadata handling, and unifies batch and streaming workloads, making it fully compatible with Apache Spark APIs.

Data Warehouse Challenges

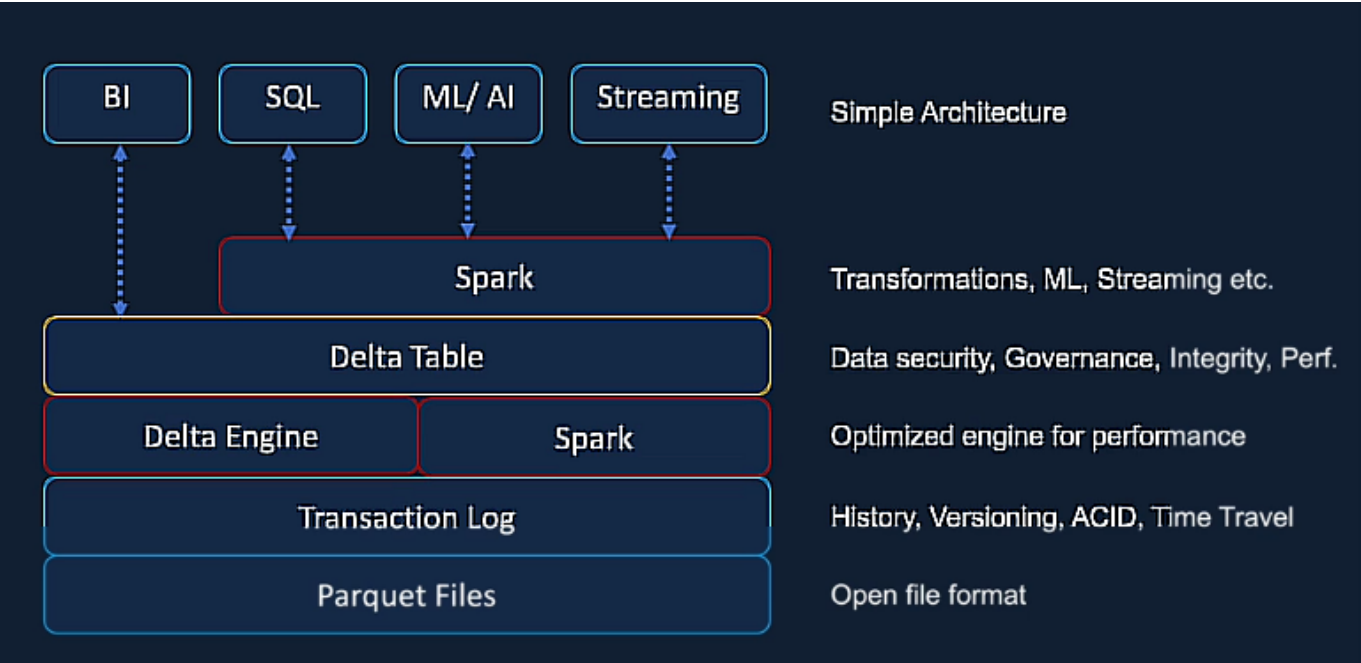
Data Warehouses, popular in the 1980s, were designed to handle structured data for business intelligence (BI) purposes. However, they struggled with scalability, high costs, lack of support for unstructured data, and slow data processing for modern workloads like machine learning (ML) and AI.

Data Lake Benefits and Pitfalls

Data Lakes, introduced around 2011, solved some of these issues by supporting structured, semi-structured, and unstructured data. They offered cheap storage and compatibility with open-source file formats like Parquet and ORC. However, they lacked ACID transactions, data governance, and performance for interactive queries, leading to unreliable data swamps.

Delta Lake and Lakehouse Architecture

Delta Lake addresses the challenges of Data Lakes by adding ACID transaction support, data versioning, and governance. This evolution gives rise to the Lakehouse architecture, which combines the strengths of both Data Warehouses and Data Lakes to handle BI, data science, and ML workloads in a unified platform.



Delta Lake Advanced Features

- **Viewing Table History:**
You can use the `DESCRIBE HISTORY` command to see the history of changes made to a table, including the version, the time of change, and details of the modifications (e.g., inserted or updated records).

- **Time Travel:**

You can retrieve past data by querying a table using a specific version number or timestamp. SQL and PySpark methods allow you to "time travel" and inspect data at any point in time.

- **GDPR Compliance with Data Deletion:**

To comply with legal requirements like GDPR, Delta Lake provides the **VACUUM** command to permanently delete old data. By default, **VACUUM** retains data for 7 days, but you can override this to delete it immediately.

- **Restoring Deleted Records:**

If records are mistakenly deleted or corrupted, Delta Lake allows you to restore them using a **MERGE** statement based on the table's history.

Transaction Logs in Delta Lake

- **Hive Meta Store Limitations:**

The Hive Meta Store only stores basic table information (name, attributes, etc.). It does not store transaction logs because it would be inefficient to retrieve that data from Hive Meta Store.

- **Delta Log:**

Delta Lake keeps transaction logs in a folder called **delta_log**, located in the table directory. These logs track all operations like table creation and data modifications. Each transaction generates a new JSON file in this folder.

- **Versioning:**

Each time data is inserted or deleted, a new version is created

The transaction log tracks these operations and the corresponding Parquet files added or removed.

- **Efficient Data Reading:**

When querying a Delta Lake table, it checks the latest transaction log to identify which Parquet files to read. For time travel, it only reads the files corresponding to the specific version.

- **Checkpoint Files:**

To improve efficiency, Delta Lake creates checkpoint files after every 10 transactions. These files summarize previous log entries, allowing the system to avoid reading all transaction logs when accessing recent data.

- **Log Retention:**

Transaction logs are retained for 30 days by default. You can adjust this retention period, but keeping logs for too long can impact performance and storage costs.

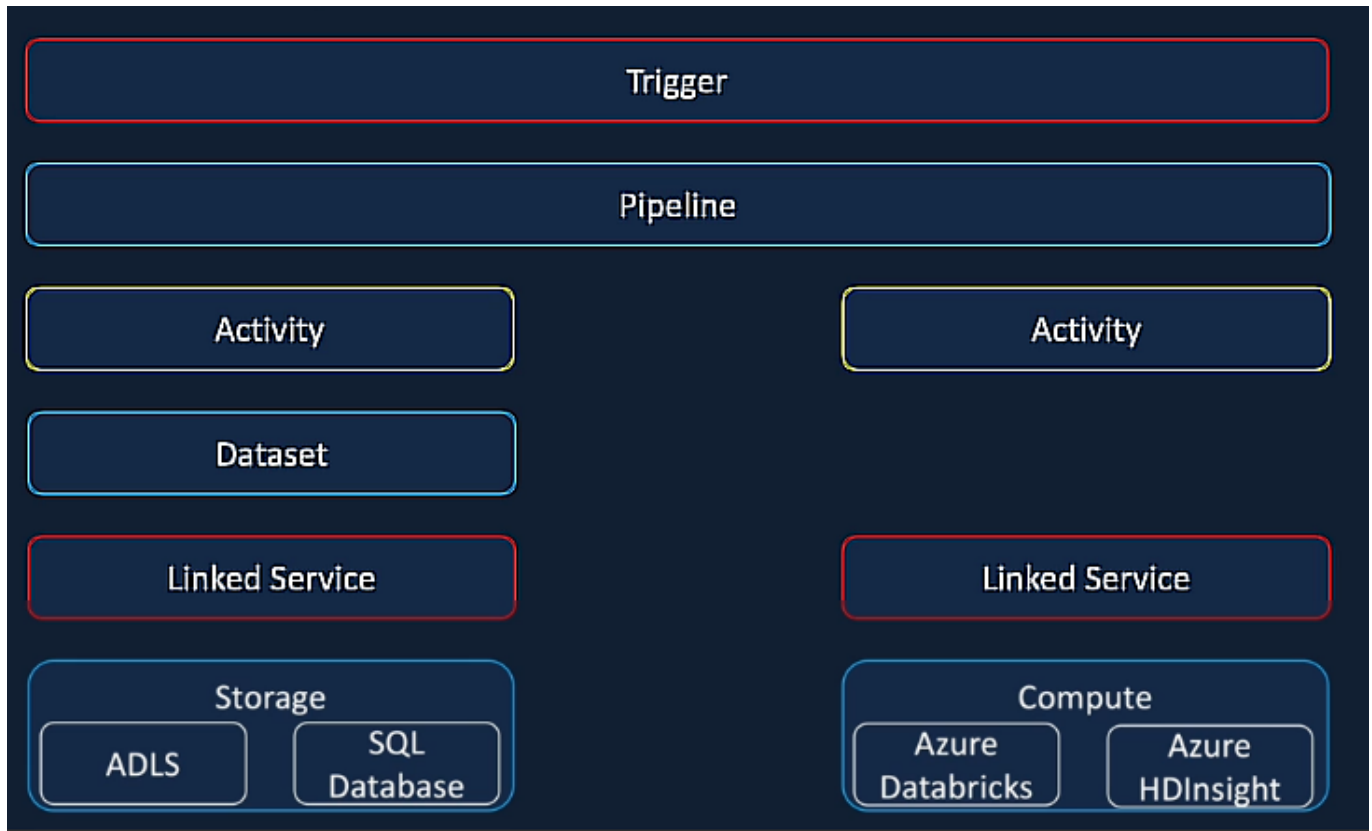
Azure Data Factory

Introduction

Azure Data Factory is a cloud service that helps you bring in, prepare, and transform data from different sources. While Data Factory has some built-in tools for ETL processes, it's not always powerful enough for more complex tasks. In those cases, It is advisable to use other tools like Azure Databricks or Spark, while Data Factory can manage the flow and orchestration of those tasks.

Data Factory doesn't store any data; it processes and moves it to another storage service, like Azure SQL Database or Azure Data Lake.

Azure Data Factory Components



- **Linked Service:** Connects ADF to external resources like storage (ADLS, SQL, etc.) or compute (Databricks, HDInsight).
- **Dataset:** Defines the structure and details of the data, such as file names and formats.
- **Activity:** Performs tasks like running a Databricks notebook or copying data. ADF also has built-in activities using its serverless compute.
- **Pipeline:** A collection of activities that can be executed in sequence or in parallel.
- **Trigger:** Automates the execution of pipelines based on schedules or events.

These components work together to create, manage, and execute data integration processes in ADF.